

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..22

PROFILE PHOTOS

 Copy as Markdown

Manage user profile photos

Add profile photo upload and display to your Serverpod app using the authentication module's built-in image handling. When you're done, signed-in users can upload a photo from Flutter, and your app displays it from a public URL.

Profile photos are stored in **public** storage, square-cropped and resized on the server, and exposed as a public HTTP URL on `UserProfileModel.imageUrl`. The authentication module registers a read-only `userProfileInfo` endpoint by default. You expose the upload methods yourself.

Before you start

- [Authentication is set up](#) and users can sign in.
- Users have a `UserProfile` (created automatically by most identity providers on first sign-in).
- You have run `serverpod generate` at least once, or started the server with `serverpod start`.

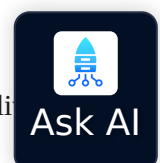
For the upload UI, add these Flutter dependencies:

pubspec.yaml

```
dependencies:
  image_picker: ^1.0.0
  image: ^4.0.15
  file_picker: '≥8.1.0 <11.0.0' # web
  image_cropper: '≥7.0.0 <13.0.0' # mobile, optional
```

Expose the profile edit endpoint

The authentication module registers `userProfileInfo` (read-only `get()`). Upload methods live on `UserProfileEditBaseEndpoint`, which you must expose on your server.



Create a concrete endpoint:

```
lib/src/endpoints/user_profile_endpoint.dart
```

```
import 'package:serverpod/serverpod.dart';
import 'package:serverpod_auth_idp_server/core.dart';

/// Endpoint to view and edit the signed-in user's profile.
class UserProfileEndpoint extends UserProfileEditBaseEndpoint {}
```

Run `serverpod generate`, or let `serverpod start` do it for you. Your app then calls the methods on `client.userProfile` (the accessor matches your endpoint class name):

Method	Returns	Auth required
<code>get()</code>	<code>UserProfileModel</code>	Yes
<code>setUserImage(ByteData image)</code>	<code>UserProfileModel</code>	Yes
<code>removeUserImage()</code>	<code>UserProfileModel</code>	Yes
<code>changeUserName(String? userName)</code>	<code>UserProfileModel</code>	Yes
<code>changeFullName(String? fullName)</code>	<code>UserProfileModel</code>	Yes

```
final profile = await client.userProfile.setUserImage(byteData);
print(profile.imageUrl); // Uri?, the public URL of the new image
```

To fetch the profile without exposing edit methods, use the module endpoint:

```
final profile = await client.modules.serverpod_auth_core.userProfileInfo.get();
```

Restrict who can edit

The built-in endpoint always acts on the signed-in user, so any signed-in user can edit their own photo. To add your own rules, override the method on your endpoint class. The legacy `serverpod_auth` module had a `userCanEditUserImage` flag for this; `serverpod_auth_idp` does not.

```
class UserProfileEndpoint extends UserProfileEditBaseEndpoint {
  @override
  Future<UserProfileModel> setUserImage(Session session, ByteData image) async {
```

```

    if (!session.authenticated!.scopes.contains(MyScope.canEditProfile)) {
      throw MyForbiddenException();
    }
    return super.setUserImage(session, image);
  }
}

```

Upload a profile photo from Flutter

There is no built-in upload widget in the authentication module. Pick an image in the app, convert it to `ByteData`, and pass it to `setUserImage`.

The server decodes the image, crops it to a square, resizes it, and stores it. The format you send does not affect the stored format.

```

import 'dart:typed_data';

import 'package:file_picker/file_picker.dart';
import 'package:flutter/foundation.dart';
import 'package:image/image.dart' as img;
import 'package:image_picker/image_picker.dart';

/// Picks an image, optionally resizes it, and returns [ByteData] for upload.
/// The server crops to square and resizes again. Client-side prep is for UX and
/// bandwidth.
Future<ByteData?> pickProfileImageBytes({int maxSize = 512}) async {
  Uint8List? rawBytes;

  if (kIsWeb) {
    final result = await FilePicker.platform.pickFiles(
      type: FileType.custom,
      allowedExtensions: ['jpg', 'jpeg', 'png'],
    );
    rawBytes = result?.files.first.bytes;
  } else {
    final picked = await ImagePicker().pickImage(source: ImageSource.gallery);
    if (picked != null) rawBytes = await picked.readAsBytes();
  }

  if (rawBytes == null) return null;

  final decoded = img.decodeImage(rawBytes);
  if (decoded == null) return null; // invalid image; show an error to the user

  final resized = decoded.width > maxSize || decoded.height > maxSize
    ? img.copyResizeCropSquare(decoded, size: maxSize)
    : decoded;

  final encoded = img.encodePng(resized);
  return ByteData.view(Uint8List.fromList(encoded).buffer);
}

```

Call the endpoint after picking:

```
Future<UserProfileModel?> uploadProfilePhoto(Client client) async {  
  final bytes = await pickProfileImageBytes();  
  if (bytes == null) return null;  
  return client.userProfile.setUserImage(bytes);  
}
```

On iOS and Android, use `image_cropper` with a circular crop before upload for better UX. Cropping is not available on web with this pattern.

Error handling

Failure	What happens
User not signed in	The call fails and surfaces as <code>ServerpodClientUnauthorized</code> in the app
No profile exists	The server throws <code>UserProfileNotFoundException</code> . The app sees a generic server error
Invalid or corrupt image bytes	Server throws when image decode fails
User cancels picker	Return <code>null</code> in the app. No server call is made

Display the profile photo

The `UserProfileModel.imageUrl` field is a public `Uri?`. When it is `null`, show a placeholder.

```
final profile = await client.userProfile.get();  
// Or: await client.modules.serverpod_auth_core.userProfileInfo.get();  
  
final imageUrl = profile?.imageUrl?.toString();  
  
CircleAvatar(  
  backgroundImage: imageUrl != null ? NetworkImage(imageUrl) : null,  
  child: imageUrl == null ? const Icon(Icons.person) : null,  
)
```

For caching, use `cached_network_image` or `extended_image`.

The built-in endpoint methods always act on the signed-in user, including the read-only `get`. To show another user's photo, fetch their `UserProfileModel` from your own endpoint and display its `imageUrl` the same way. The URL itself is not access-controlled, so anything holding it can load the image.

Remove or replace a photo

Remove the photo (`imageUrl` becomes `null`):

```
final profile = await client.userProfile.removeUserImage();
```

Unlike `legacy serverpod_auth`, removing a photo does **not** restore a generated default avatar. Show a placeholder in your UI, or call `setDefaultUserImage` on signup (see below).

Replace a photo by calling `setUserImage` again with new bytes. The server creates a new stored file with a new random suffix.

Configure image size and format

Pass `UserProfileConfig` to `initializeAuthServices` in `server.dart`:

Setting	Default	Description
<code>userImageSize</code>	256	Output width and height in pixels (square)
<code>userImageFormat</code>	<code>UserProfileImageType.jpg</code>	Stored format (<code>.jpg</code> or <code>.png</code>)
<code>userImageQuality</code>	70	JPG quality (ignored for PNG)
<code>userImageGenerator</code>	<code>defaultUserImageGenerator</code>	Used by <code>setDefaultUserImage()</code>
<code>imageFetchFunc</code>	<code>http.get</code>	Used when importing from a URL (social sign-in)

```
pod.initializeAuthServices(  
  userProfileConfig: UserProfileConfig(  
    userImageSize: 512,  
    userImageFormat: UserProfileImageType.png,  
    userImageQuality: 85,  
  ),  
);
```

```

userImageGenerator: defaultUserImageGenerator,
onAfterUserProfileCreated: (session, profile, {required transaction}) async {
  await AuthService.instance.userProfiles.setDefaultUserImage(
    session,
    profile.authUserId,
    transaction: transaction,
  );
},
),
identityProviderBuilders: [...],
tokenManagerBuilders: [...],
);

```

The `defaultUserImageGenerator` function produces a solid-colored square with the first letter of the user name. It looks circular when you display it in a circular widget such as `CircleAvatar`.

The server automatically validates and optimizes uploads: it decodes bytes (must be a valid image), square-crops to `userImageSize`, re-encodes as JPG or PNG per config, and stores the file in public storage.

Recommended checks in the app (not enforced by the server):

- Allow only JPG and PNG extensions.
- Reject files over roughly 5 to 10 MB before upload.
- Verify `decodeImage` succeeds before calling `setUserImage`.

Providers such as Google import a profile photo whenever the user signs in, the provider returns a photo, and the user has no image set. This is not limited to the first sign-in, so a photo the user removed can come back on their next sign-in with that provider.

Configure storage for production

In development, profile images use the default database-backed public storage. No extra setup is required.

Public URLs look like:

```

http://localhost:8080/serverpod_cloud_storage?
method=file&path=serverpod/user_images/{authUserId}-{id}.jpg

```

Set `publicHost`, `publicPort`, and `publicScheme` in `config/development.yaml` to match how clients reach your API server.

For production, configure object storage (S3, Google Cloud Storage, or R2) for `storageId: 'public'`. See [Uploading files](#). Profile images require publicly accessible URLs because clients load them directly over HTTP.

Use server-side APIs for custom logic

In endpoints or callbacks, use `AuthServices.instance.userProfiles`:

```
import 'package:serverpod_auth_idp_server/core.dart';

// Set from bytes (same as the endpoint)
await AuthServices.instance.userProfiles.setUserImageFromBytes(
  session, authUserId, imageBytes,
);

// Import from a URL (e.g. after OAuth)
await AuthServices.instance.userProfiles.setUserImageFromUrl(
  session, authUserId, Uri.parse('https://example.com/photo.jpg'),
);

// Generate and store a default avatar
await AuthServices.instance.userProfiles.setDefaultUserImage(session, authUserId);

// Remove image
await AuthServices.instance.userProfiles.removeUserImage(session, authUserId);

// Read profile in any endpoint
final profile = await session.authenticated!.userProfile(session);
```

Verify

1. Sign in to your app.
2. Call `client.userProfile.setUserImage(byteData)` with a test PNG or JPG.
3. Confirm the returned `imageUrl` is non-null and starts with your server's public host.
4. Open `imageUrl` in a browser. The image loads.
5. Confirm your UI updates after upload.
6. Call `removeUserImage()`. The `imageUrl` field becomes `null` and your placeholder appears.

Troubleshooting

`setUserImage` is missing on the client

Create `UserProfileEndpoint` extends `UserProfileEditBaseEndpoint` and run `serverpod generate`.

`UserProfileNotFoundException` in the server logs

The signed-in user has no profile yet. Ensure your identity provider creates one on first sign-in. The app only sees a generic server error, so check the server logs for this exception.

Image URL does not load

Check that `publicHost`, `publicPort`, and `publicScheme` under `apiServer` in your server config match how apps reach the API server.

Invalid image error

The bytes are corrupt or not an image. Validate with `decodeImage` on the client before upload.

Image looks low quality

Default JPG quality is 70. Raise `userImageQuality` or use PNG format in `UserProfileConfig`.

Web upload fails silently

The `file_picker` package may return null bytes. Ensure you read bytes correctly from the picked file.

Related

- [Working with users](#): profiles, names, and the edit endpoint overview
- [Authentication setup](#): initial configuration of the authentication module
- [Uploading files](#): cloud storage for production
- [Legacy: displaying or editing user images](#): legacy `serverpod_auth` widgets